

Name : AKASH SAMPATH

Module : Research Questions Part 2 Recent

Q1. What is ORM (object relationship mapping) and what is Hibernate in Java?

Term	Meaning	Example	Concept
Object	Java Object contains data and behavior	Customer customer = new Customer();	Java works with objects
Relational Database	Database that stores data in tables , rows and columns	MYSQL customer table	MYSQL works with tables
ORM	Connects and maps java objects to database tables	Customer <-> customer table	Object <-> Table
Object Relational Mapping	Full form of ORM	customer.name – customer.name column	Maps java world to DB world
WHY ORM	Avoid writing repetitive database code manually	Instead of manually doing for every operation	Makes database work easier
JPA	Standard specification that defines how java ORM should work	@Entity , @Id , @OneToMany	JPA = Rules
Hibernate	Framework that implements JPA and performs ORM work	Hibernate converts java operations into SQL	Hibernate = Implementation
JPA vs Hibernate	JPA defines the rules . Hibernate implements them	JPA says how ORM should work and Hibernate does the work	JPA = What , Hibernate = How
Entity	Java class that represents a database table	@Entity class Customer	Class – Table

@Entity	Tells Hibernate that a Java class should be mapped to a DB table	@Entity public class Customer	Hibernate manage this class
@Id	Identifies the primary key	@Id private Long id;	@Id = Primary key
@GeneratedValue	Allows database and framework to generate IDs	@GeneratedValue(...)	Automatically creates IDs
@Column	Configures a database column	@Column(name="consumer_number")	Java field – DB column
Relationship	Defines how database entities are connected	Customer has many bills	Table can be related
@OneToMany	One object has many object	One customer has many bills	1 – Many
@ManyToOne	Many objects belong to one object	Many bills one customer	Many - 1
@ManyToMany	Many objects relate to many objects	Students – Courses	Many – Many
Repository	Java interface used to perform database operations	Customer Repository	Your DB access layer

Spring Data JPA	Makes repository and database operations much easier	JpaRepository<Customer, Long>	Less code for DB operations
save()	Saves an object into the database	customerRepository.save(customer)	Usually becomes INSERT/UPDATE
findById()	Finds one record using its ID	findById	Usually becomes SELECT
findAll()	Gets all record	customerRepository.findAll()	Gets all rows
deleteById()	Deletes a record	deleteById	Usually becomes DELETE
SQL	Language used to communicate with relational database	SELECT * FROM customer	Database understands SQL
JDBC	Java technology used to communicate with databases	Java → JDBC → MySQL	Low level DB communication
Hibernate Job	Converts and manages java object and operations and database interaction	Java save() → SQL INSERT	JAVA – SQL translator / manager
Example Electricity Bill	Java [Electricity Bill] maps to DB electricity_bill	ElectricityBill ↔ electricity_bil	Project is a perfect ORM
Complete flow	Full path from application to database	React → Spring Boot → Repository → JPA → Hibernate → JDBC → MySQL	

Core Concepts :

ORM	Object Database Mapping
JPA	ORM specification or rules
Hibernate	JPA implementation and ORM framework
JDBC	Java Database Communication
MYSQL	Actual Database

Example :

Java

MYSQL

Customer	customer table
ElectricityBill	electricity_bill table
customer.id	customer.id
bill.amount	bill.amount
bill.customer	customer_id

So therefore ,

ORM maps Java objects to database tables, JPA defines the ORM rules, and Hibernate implements those rules to communicate with the database.

Q2.What is the equivalent of this in Python?

As Hibernate in Java is similar to SQL Alchemy in Python and as they not literally identical. SQL Alchemy is a broader SQL database toolkit with ORM capabilities while Hibernate is known as a Java ORM framework

Suppose you MSQl database contains

```
customer
-----
id | name | email
-----
1 | Akash | akash@gmail.com
```

In Java / Hibernate

```
@Entity
public class Customer {

    @Id
    private Long id;

    private String name;
    private String email;
}
```

In Python / SQL Alchemy

```
class Customer(Base):  
  
    __tablename__ = "customer"  
  
    id = Column(Integer, primary_key=True)  
    name = Column(String)  
    email = Column(String)
```

So therefore the idea is the same like

Java

Python

Customer class	Customer class
@Entity	SQL Alchemy model
@Id	primary_key=True
Java object	Python Object
Hibernate	SQL Alchemy
MYSQL Table	MYSQL table

Reading a customer : Example :

Java	customerRepository.findById(1L);
Python	customer = session.get(Customer, 1)
SQL Conceptually do both	SELECT * FROM customer WHERE id = 1;

Therefore ,

Java uses JPA/Hibernate for ORM; Python commonly uses SQLAlchemy (or Django ORM) for ORM. Both allow you to work with database data as programming-language objects instead of manually handling every SQL operation.

Q3. What is CRUD in general in web application and how is it related to HTTP methods? What is API and what is webAPI how is webapi related to HTTP ?

Flow :

Web Application	Complete application users interact with
Frontend = React	What the users sees
Backend = Spring Boot	Processes requests
Database = MySQL	Stores data
API / WEB API	Communication interface between applications
HTTP	Communication protocol used by Web APIs
CRUD	Create , Read , Update , Delete

Want to do	CRUD	HTTP method	Example
Add a new bill	CREATE	POST	POST /api/bills
Get all bills	READ	GET	GET /api/bills
Get one bill	READ	GET	GET /api/bills/101
Change a bill	UPDATE	PUT	PUT /api/bills/101
Change a part of a bill	UPDATE	PATCH	PATCH /api/bills/101
Remove a bill	DELETE	DELETE	DELETE /api/bills/101